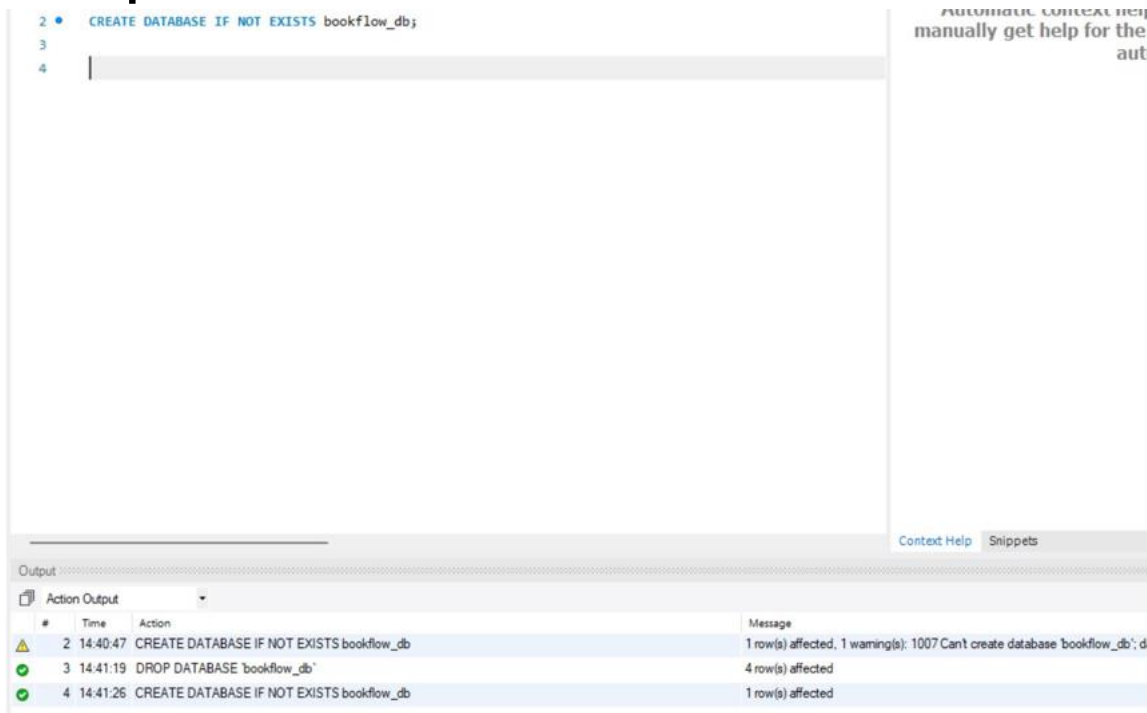


2520030178 – Aasritha Kandepu

WEEK-2 PRACTICAL DATABASE SYSTEM ENGINEERING

Step 1 — Create Database CREATE DATABASE bookflow_db; USE bookflow_db;

output:



Step 2 — Create Books Table (DDL + Constraints) CREATE TABLE Books (book_id INT PRIMARY KEY, title VARCHAR(100) NOT NULL, isbn VARCHAR(20) UNIQUE, published_year INT CHECK (published_year < 2027));

output:

```
2 • CREATE DATABASE IF NOT EXISTS bookflow_db;
3 • USE bookflow_db;
4 • CREATE TABLE books (
5     book_id
6     INT AUTO_INCREMENT PRIMARY KEY,
7     title
8     VARCHAR(255) NOT NULL,
9     isbn
10    VARCHAR(13) NOT NULL UNIQUE,
11    published_year INT,
12    CONSTRAINT chk_published_year CHECK (published_year < 2027)
13 );
14
15
```

Automat
manually go

Context Help Snipp

Output

Action Output

#	Time	Action	Message
✓ 4	14:41:26	CREATE DATABASE IF NOT EXISTS bookflow_db	1 row(s) affected
✓ 5	14:48:14	USE bookflow_db	0 row(s) affected
✓ 6	14:49:23	CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255) ...	0 row(s) affected

Step 3 & 4 — Insert Books and Display INSERT INTO Books (book_id, title, isbn, published_year) VALUES (1, 'The Great Gatsby', '9780743273565', 1925), (2, 'To Kill a Mockingbird', '9780061120084', 1960), (3, '1984', '9780451524935', 1949);
SELECT * FROM Books;

output:

2 • CREATE DATABASE IF NOT EXISTS bookflow_db;
3 • USE bookflow_db;
4 • CREATE TABLE books (
5 book_id
6 INT AUTO_INCREMENT PRIMARY KEY,
7 title
8 VARCHAR(255) NOT NULL,
9 isbn
10 VARCHAR(13) NOT NULL UNIQUE,
11 published_year INT,
12 CONSTRAINT chk_published_year CHECK (published_year < 2027)
13);
14 • CREATE TABLE members (
15 member_id INT AUTO_INCREMENT PRIMARY KEY,
16 full_name VARCHAR(100) NOT NULL,
17 email
18 VARCHAR(150) NOT NULL UNIQUE
19);
20

manua

Context Help

Output

Action Output

#	Time	Action	Message
5	14:48:14	USE bookflow_db	0 row(s) affected
6	14:49:23	CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255) ...	0 row(s) affected
7	14:50:43	CREATE TABLE members (member_id INT AUTO_INCREMENT PRIMARY KEY, full_name VARCHAR(100)...	0 row(s) affected

step 5, 6 & 7 — Members Table: Create, Insert, Display
CREATE TABLE Members (member_id INT PRIMARY KEY, full_name VARCHAR(100), email VARCHAR(100) UNIQUE);
INSERT INTO Members (member_id, full_name, email) VALUES (101, 'John Smith', 'john.smith@email.com'), (102, 'Emma Wilson', 'emma.wilson@email.com'), (103, 'Michael Brown', 'michaelS.brown@email.com');
SELECT * FROM Members;

output:

Automatic context help manually get help for t

```

46 • CREATE TABLE Members (
47     member_id INT PRIMARY KEY,
48     full_name VARCHAR(100),
49     email VARCHAR(100) UNIQUE
50 );
51 • INSERT INTO Members (member_id, full_name, email) VALUES
52     (101, 'John Smith', 'john.smith@email.com'),
53     (102, 'Emma Wilson', 'emma.wilson@email.com'),
54     (103, 'Michael Brown', 'michael.brown@email.com');
55 • SELECT * FROM Members;

```

member_id	full_name	email
1	Anil Kumar	anil.kumar@example.com
2	Priya Sharma	priya.sharma@example.com
3	Ravi Verma	ravi.verma@example.com
101	John Smith	john.smith@email.com
102	Emma Wilson	emma.wilson@email.com
103	Michael Brown	michael.brown@email.com
NULL	NULL	NULL

Members 3 x Apply Context Help Snippets

Output

Action Output

#	Time	Action	Message
23	15:15:25	CREATE TABLE Members (member_id INT PRIMARY KEY, full_name VARCHAR(100), email VARCHAR(10...	Error Code: 1050. Table 'members' already exists
24	15:15:32	INSERT INTO Members (member_id, full_name, email) VALUES (101, 'John Smith', 'john.smith@email.com'), (...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
25	15:15:32	SELECT * FROM Members LIMIT 0, 1000	6 row(s) returned

Step 8 — Create Loans Table (FOREIGN KEYS) CREATE TABLE Loans (loan_id INT PRIMARY KEY, member_id INT, book_id INT, loan_date DATE, FOREIGN KEY (member_id) REFERENCES Members(member_id), FOREIGN KEY (book_id) REFERENCES Books(book_id));

output:

exp1* customer_transactions SQL File 4* SQL File 5* SQL File 6* x bookflow_db - Schema SQL File 7* SQLAdditions

Limit to 1000 rows

```

42 (2, 'To Kill a Mockingbird', '9780061120084', 1960),
43 (3, '1984', '9780451524935', 1949);
44 • SELECT * FROM Books;
45
46 • CREATE TABLE Members (
47     member_id INT PRIMARY KEY,
48     full_name VARCHAR(100),
49     email VARCHAR(100) UNIQUE
50 );
51 • INSERT INTO Members (member_id, full_name, email) VALUES
52 (101, 'John Smith', 'john.smith@email.com'),
53 (102, 'Emma Wilson', 'emma.wilson@email.com'),
54 (103, 'Michael Brown', 'michael.brown@email.com');
55 • SELECT * FROM Members;
56
57 • CREATE TABLE Loans (
58     loan_id INT PRIMARY KEY,
59     member_id INT,
60     book_id INT,
61     loan_date DATE,
62     FOREIGN KEY (member_id) REFERENCES Members(member_id),
63     FOREIGN KEY (book_id) REFERENCES Books(book_id)
64 );

```

Context Help

Output

Action Output

#	Time	Action	Message
✓ 24	15:15:32	INSERT INTO Members (member_id, full_name, email) VALUES (101, 'John Smith', 'john.smith@email.com'), (...	3 row(s) affected Records: 3 Duplicates: 0 Wa
✓ 25	15:15:32	SELECT * FROM Members LIMIT 0, 1000	6 row(s) returned
✓ 26	15:21:33	CREATE TABLE Loans (loan_id INT PRIMARY KEY, member_id INT, book_id INT, loan_date DATE, FO...	0 row(s) affected

Step 9 & 10 — Insert 10 Loans and Display INSERT INTO Loans (loan_id, member_id, book_id, loan_date) VALUES (1, 101, 1, '2025-01-05'), (2, 102, 2, '2025-01-08'), (3, 103, 3, '2025-01-10'), (4, 101, 2, '2025-02-01'), (5, 102, 1, '2025-02-05'), (6, 103, 2, '2025-02-12'), (7, 101, 3, '2025-03-01'), (8, 102, 3, '2025-03-07'), (9, 103, 1, '2025-03-15'), (10, 101, 1, '2025-04-01'); SELECT * FROM Loans;

output:

exp1* customer_transactions SQL File 4* SQL File 5* SQL File 6* x bookflow_db - Schema SQL File 7* SQLAdditions

Limit to 1000 rows

```

63 FOREIGN KEY (book_id) REFERENCES Books(book_id)
64 );
65
66 • INSERT INTO Loans (loan_id, member_id, book_id, loan_date) VALUES
67 (1, 101, 1, '2025-01-05'), (2, 102, 2, '2025-01-08'),
68 (3, 103, 3, '2025-01-10'), (4, 101, 2, '2025-02-01'),
69 (5, 102, 1, '2025-02-05'), (6, 103, 2, '2025-02-12'),
70 (7, 101, 3, '2025-03-01'), (8, 102, 3, '2025-03-07'),
71 (9, 103, 1, '2025-03-15'), (10, 101, 1, '2025-04-01');
72 • SELECT * FROM Loans;

```

Result Grid

	loan_id	member_id	book_id	loan_date
▶	1	101	1	2025-01-05
	2	102	2	2025-01-08
	3	103	3	2025-01-10
	4	101	2	2025-02-01
	5	102	1	2025-02-05
	6	103	2	2025-02-12
	7	101	3	2025-03-01
	8	102	3	2025-03-07
	9	103	1	2025-03-15
	10	101	1	2025-04-01
*	NULL	NULL	NULL	NULL

Loans 4 x

Output

Action Output

#	Time	Action	Message
✓ 26	15:21:33	CREATE TABLE Loans (loan_id INT PRIMARY KEY, member_id INT, book_id INT, loan_date DATE, FO...	0 row(s) affected
✓ 27	15:22:33	INSERT INTO Loans (loan_id, member_id, book_id, loan_date) VALUES (1, 101, 1, '2025-01-05'), (2, 102, 2, ...	10 row(s) affected Records: 10 Duplicates: 0 W
✓ 28	15:22:33	SELECT * FROM Loans LIMIT 0, 1000	10 row(s) returned

Step 11 — JOIN Query (Who Borrowed What) SELECT m.full_name AS Member_Name, b.title AS Book_Title FROM Loans l INNER JOIN Members m ON l.member_id = m.member_id INNER JOIN Books b ON l.book_id = b.book_id;

output:

exp1* customer_transactions SQL File 4* SQL File 5* SQL File 6* x bookflow_db - Schema SQL File 7* SQLAdditions

Limit to 1000 rows

```

68 (3, 103, 3, '2025-01-10'), (4, 101, 2, '2025-02-01'),
69 (5, 102, 1, '2025-02-05'), (6, 103, 2, '2025-02-12'),
70 (7, 101, 3, '2025-03-01'), (8, 102, 3, '2025-03-07'),
71 (9, 103, 1, '2025-03-15'), (10, 101, 1, '2025-04-01');
72 • SELECT * FROM Loans;
73
74 • SELECT m.full_name AS Member_Name, b.title AS Book_Title
75 FROM Loans l
76 INNER JOIN Members m ON l.member_id = m.member_id
77 INNER JOIN Books b ON l.book_id = b.book_id;

```

Automatic context help manually get help for t

Result Grid

Member_Name	Book_Title
John Smith	The Alchemist
Emma Wilson	The Alchemist
Michael Brown	The Alchemist
John Smith	The Alchemist
Emma Wilson	Clean Code
John Smith	Clean Code
Michael Brown	Clean Code
Michael Brown	Atomic Habits
John Smith	Atomic Habits
Emma Wilson	Atomic Habits

Result 5 x Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message
27	15:22:33	INSERT INTO Loans (loan_id, member_id, book_id, loan_date) VALUES (1, 101, 1, '2025-01-05'), (2, 102, 2, ...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
28	15:22:33	SELECT * FROM Loans LIMIT 0, 1000	10 row(s) returned
29	15:23:34	SELECT m.full_name AS Member_Name, b.title AS Book_Title FROM Loans l INNER JOIN Members m ON l...	10 row(s) returned

Step 12 — GROUP BY Query (Books per Year) SELECT published_year, COUNT(book_id) AS Total_Books FROM Books GROUP BY published_year ORDER BY published_year;

output:

exp1* customer_transactions SQL File 4* SQL File 5* SQL File 6* x bookflow_db - Schema SQL File 7* SQLAdditions

Limit to 1000 rows

```

73
74 • SELECT m.full_name AS Member_Name, b.title AS Book_Title
75 FROM Loans l
76 INNER JOIN Members m ON l.member_id = m.member_id
77 INNER JOIN Books b ON l.book_id = b.book_id;
78
79 • SELECT published_year, COUNT(book_id) AS Total_Books
80 FROM Books
81 GROUP BY published_year
82 ORDER BY published_year;

```

Automatic (manually get f

Result Grid

	published_year	Total_Books
▶	1988	1
	2008	1
	2018	1

Export: | Wrap Cell Content: |

Result 6 x

Read Only Context Help Snippets

Output

Action Output

#	Time	Action	Message
✓ 28	15:22:33	SELECT * FROM Loans LIMIT 0, 1000	10 row(s) returned
✓ 29	15:23:34	SELECT m.full_name AS Member_Name, b.title AS Book_Title FROM Loans l INNER JOIN Members m ON l...	10 row(s) returned
✓ 30	15:24:39	SELECT published_year, COUNT(book_id) AS Total_Books FROM Books GROUP BY published_year ORD...	3 row(s) returned

Step 13–16 — Donation via TRANSACTION (Atomicity) CREATE TABLE Donation_History (donation_id INT PRIMARY KEY, book_id INT, donor_name VARCHAR(100), donation_date DATE, FOREIGN KEY (book_id) REFERENCES Books(book_id)); START TRANSACTION; INSERT INTO Books (book_id, title, isbn, published_year) VALUES (4, 'Animal Farm', '9780451526342', 1945); INSERT INTO Donation_History (donation_id, book_id, donor_name, donation_date) VALUES (1, 4, 'Raj Kumar', CURDATE()); COMMIT;

output:

exp1* customer_transactions SQL File 4* SQL File 5* SQL File 6* x bookflow_db - Schema SQL File 7* SQLAdditions

Limit to 1000 rows

```

79 • SELECT published_year, COUNT(book_id) AS Total_Books
80 FROM Books
81 GROUP BY published_year
82 ORDER BY published_year;
83
84 • CREATE TABLE Donation_History (
85     donation_id INT PRIMARY KEY,
86     book_id INT,
87     donor_name VARCHAR(100),
88     donation_date DATE,
89     FOREIGN KEY (book_id) REFERENCES Books(book_id)
90 );
91
92 • START TRANSACTION;
93
94 • INSERT INTO Books (book_id, title, isbn, published_year)
95 VALUES (4, 'Animal Farm', '9780451526342', 1945);
96
97 • INSERT INTO Donation_History (donation_id, book_id, donor_name, donation_date)
98 VALUES (1, 4, 'Raj Kumar', CURDATE());
99
100 • COMMIT;
101

```

Automatic content manually get help for

Context Help Snippets

Output

Action Output

#	Time	Action	Message
33	15:25:58	INSERT INTO Books (book_id, title, isbn, published_year) VALUES (4, 'Animal Farm', '9780451526342', 1945)	1 row(s) affected
34	15:25:58	INSERT INTO Donation_History (donation_id, book_id, donor_name, donation_date) VALUES (1, 4, 'Raj Kumar', CURDATE())	1 row(s) affected
35	15:25:58	COMMIT	0 row(s) affected

Step 17 & 18 — Index on ISBN + Fast Search CREATE INDEX idx_books_isbn ON Books(isbn); SELECT * FROM Books WHERE isbn = '9780451524935';

output:

AUTOMATIC CO
manually get he

```

95 VALUES (4, 'Animal Farm', '9780451526342', 1945);
96
97 • INSERT INTO Donation_History (donation_id, book_id, donor_name, donation_date)
98 VALUES (1, 4, 'Raj Kumar', CURDATE());
99
100 • COMMIT;
101
102 • CREATE INDEX idx_books_isbn ON Books(isbn);
103 • SELECT * FROM Books WHERE isbn = '9780451524935';

```

Result Grid

book_id	title	isbn	published_year
NULL	NULL	NULL	NULL

Books 7 x

Output

Action Output

#	Time	Action	Message
35	15:25:58	COMMIT	0 row(s) affected
36	15:27:29	CREATE INDEX idx_books_isbn ON Books(isbn)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
37	15:27:29	SELECT * FROM Books WHERE isbn = '9780451524935' LIMIT 0, 1000	0 row(s) returned

Assignments

Problem Statement 1: SkyTrack Airline Management System A regional airline company currently manages its flight schedules, passenger records, and ticket bookings using spreadsheets. As the number of flights and passengers continues to grow, it has become difficult to track reservations, maintain accurate records, and generate operational reports. To improve efficiency, the airline plans to develop a database system called SkyTrack. As a database developer, your task is to create a database named skytrack_db and design three tables: Flights, Passengers, and Bookings. The Flights table should contain details such as flight_id, flight_number, source, destination, departure_date, and ticket_price, while the Passengers table should store passenger_id, passenger_name, and email. The Bookings table should connect passengers and flights using passenger_id and flight_id as foreign keys. Apply appropriate constraints, including PRIMARY KEY, UNIQUE, and CHECK constraints to ensure data integrity. Insert at least 10 records into each table. Display all table contents and write an SQL query using INNER JOIN to show the passenger name, flight number, source, and

destination for every booking. Generate a report showing the total number of flights operating to each destination using COUNT() and GROUP BY. Create a Flight_History table and demonstrate the use of a Transaction by adding a new flight and simultaneously recording the entry in the history table, ensuring both operations succeed or fail together. Finally, create an Index on the flight_number column and explain why searching by flight number is faster than searching by destination. This exercise develops skills in relational database design, JOIN operations, aggregate functions, transactions, and indexing.

```
output:CREATE DATABASE skytrack_db;
```

```
USE skytrack_db;
```

```
-- 1. Flights Table
```

```
CREATE TABLE Flights (  
    flight_id INT PRIMARY KEY,  
    flight_number VARCHAR(20) UNIQUE NOT NULL,  
    source VARCHAR(50) NOT NULL,  
    destination VARCHAR(50) NOT NULL,  
    departure_date DATE NOT NULL,  
    ticket_price DECIMAL(10,2) CHECK (ticket_price > 0)  
);
```

```
-- 2. Passengers Table
```

```
CREATE TABLE Passengers (  
    passenger_id INT PRIMARY KEY,  
    passenger_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL
```

);

-- 3. Bookings Table

```
CREATE TABLE Bookings (  
    booking_id INT PRIMARY KEY,  
    passenger_id INT NOT NULL,  
    flight_id INT NOT NULL,  
    booking_date DATE NOT NULL,  
    FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id),  
    FOREIGN KEY (flight_id) REFERENCES Flights(flight_id)  
);
```

-- Insert 10 Flights

```
INSERT INTO Flights VALUES  
(1, 'ST101', 'Hyderabad', 'Delhi', '2026-09-01', 5500),  
(2, 'ST102', 'Hyderabad', 'Mumbai', '2026-09-02', 4500),  
(3, 'ST103', 'Chennai', 'Bangalore', '2026-09-03', 3000),  
(4, 'ST104', 'Delhi', 'Kolkata', '2026-09-04', 6000),  
(5, 'ST105', 'Mumbai', 'Goa', '2026-09-05', 3500),  
(6, 'ST106', 'Bangalore', 'Delhi', '2026-09-06', 5200),  
(7, 'ST107', 'Chennai', 'Hyderabad', '2026-09-07', 4000),  
(8, 'ST108', 'Kolkata', 'Mumbai', '2026-09-08', 5800),  
(9, 'ST109', 'Delhi', 'Bangalore', '2026-09-09', 5000),  
(10, 'ST110', 'Hyderabad', 'Goa', '2026-09-10', 4200);
```

-- Insert 10 Passengers

INSERT INTO Passengers VALUES

(1, 'Chaitanya Kiran', 'chaitanya@gmail.com'),
(2, 'Rahul Sharma', 'rahul@gmail.com'),
(3, 'Priya Reddy', 'priya@gmail.com'),
(4, 'Arjun Kumar', 'arjun@gmail.com'),
(5, 'Sneha Rao', 'sneha@gmail.com'),
(6, 'Vikram Singh', 'vikram@gmail.com'),
(7, 'Ananya Das', 'ananya@gmail.com'),
(8, 'Kiran Patel', 'kiran@gmail.com'),
(9, 'Meghana Devi', 'meghana@gmail.com'),
(10, 'Rohit Verma', 'rohit@gmail.com');

-- Insert 10 Bookings

INSERT INTO Bookings VALUES

(1, 1, 1, '2026-08-17'),
(2, 2, 2, '2026-08-17'),
(3, 3, 3, '2026-08-17'),
(4, 4, 4, '2026-08-18'),
(5, 5, 5, '2026-08-18'),
(6, 6, 6, '2026-08-18'),
(7, 7, 7, '2026-08-19'),
(8, 8, 8, '2026-08-19'),

```
(9, 9, 9, '2026-08-19'),  
(10, 10, 10, '2026-08-20');
```

```
-- Display all tables
```

```
SELECT * FROM Flights;
```

```
SELECT * FROM Passengers;
```

```
SELECT * FROM Bookings;
```

```
-- INNER JOIN: Passenger + Flight details
```

```
SELECT
```

```
    p.passenger_name,
```

```
    f.flight_number,
```

```
    f.source,
```

```
    f.destination
```

```
FROM Bookings b
```

```
INNER JOIN Passengers p
```

```
    ON b.passenger_id = p.passenger_id
```

```
INNER JOIN Flights f
```

```
    ON b.flight_id = f.flight_id;
```

```
-- Number of flights operating to each destination
```

```
SELECT
```

```
    destination,
```

```
    COUNT(*) AS total_flights
```


FROM Flights

GROUP BY destination;

-- Flight History Table

```
CREATE TABLE Flight_History (  
    history_id INT PRIMARY KEY AUTO_INCREMENT,  
    flight_id INT NOT NULL,  
    flight_number VARCHAR(20) NOT NULL,  
    action VARCHAR(50) NOT NULL,  
    action_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Transaction

START TRANSACTION;

INSERT INTO Flights

VALUES (11, 'ST111', 'Hyderabad', 'Pune', '2026-09-11', 4300);

INSERT INTO Flight_History

(flight_id, flight_number, action)

VALUES

(11, 'ST111', 'New Flight Added');

COMMIT;

-- Verify transaction

SELECT * FROM Flights;

SELECT * FROM Flight_History;

-- Create Index

CREATE INDEX idx_flight_number

ON Flights(flight_number);

-- Search using flight number

SELECT *

FROM Flights

WHERE flight_number = 'ST105';

The screenshot shows the MySQL Model (sakila_full.mwb) interface. The left sidebar displays a list of schemas: bank_db, bankdb, bookflow_db1, college_db, library_db, library_week1_db, and sys. The main window shows a SQL query editor with the following code:

```

118 • SELECT * FROM Flight_History;
119
120 -- Create Index
121 • CREATE INDEX idx_flight_number
122 ON Flights(flight_number);
123
124 -- Search using flight number
125 • SELECT *
126 FROM Flights
127 WHERE flight_number = 'ST105';

```

Below the query editor, the Result Grid shows the following data:

flight_id	flight_number	source	destination	departure_date	ticket_price
5	ST105	Mumbai	Goa	2026-09-05	3500.00
NULL	NULL	NULL	NULL	NULL	NULL

The bottom section of the interface shows the Output pane with the following messages:

#	Time	Action	Message
57	15:31:30	SELECT * FROM Flight_History LIMIT 0, 1000	1 row(s) returned
58	15:31:30	CREATE INDEX idx_flight_number ON Flights(flight_number)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
59	15:31:30	SELECT * FROM Flights WHERE flight_number = 'ST105' LIMIT 0, 1000	1 row(s) returned

Problem Statement 2: MediCare Hospital Management System A hospital currently maintains patient records, doctor information, and appointment schedules manually, which often results in scheduling conflicts, misplaced records, and delays in accessing important information. To modernize its operations, the hospital wants to develop a database system called MediCare HMS. As a database developer, your task is to create a database named `medicare_db` and design three tables: `Doctors`, `Patients`, and `Appointments`. The `Doctors` table should store details such as `doctor_id`, `doctor_name`, `specialization`, and `consultation_fee`, while the `Patients` table should contain `patient_id`, `patient_name`, and `email`. The `Appointments` table should connect doctors and patients using foreign keys and include an appointment date. Apply appropriate constraints such as `PRIMARY KEY`, `UNIQUE`, and `CHECK` constraints to ensure data accuracy and consistency. Insert at least 10 records into each table. Display all table contents and write an SQL query using `INNER JOIN` to show the patient name, doctor name, specialization, and appointment date for all appointments. Generate a report displaying the number of doctors available in each specialization using `COUNT()` and `GROUP BY`. Create a `Doctor_History` table and demonstrate the use of a Transaction by registering a new doctor and

simultaneously recording the registration in the history table, ensuring both operations are completed successfully as a single unit. Finally, create an Index on the specialization column and explain how indexing improves search performance when locating doctors by specialization. This exercise helps students understand database design, table relationships, SQL queries, aggregate functions, transactions, and query optimization techniques.

output:

```
CREATE DATABASE medicare_db;
```

```
USE medicare_db;
```

```
-- 1. Doctors Table
```

```
CREATE TABLE Doctors (  
    doctor_id INT PRIMARY KEY,  
    doctor_name VARCHAR(100) NOT NULL,  
    specialization VARCHAR(100) NOT NULL,  
    consultation_fee DECIMAL(10,2) CHECK (consultation_fee > 0)  
);
```

```
-- 2. Patients Table
```

```
CREATE TABLE Patients (  
    patient_id INT PRIMARY KEY,  
    patient_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL  
);
```

-- 3. Appointments Table

```
CREATE TABLE Appointments (  
    appointment_id INT PRIMARY KEY,  
    doctor_id INT NOT NULL,  
    patient_id INT NOT NULL,  
    appointment_date DATE NOT NULL,  
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),  
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)  
);
```

-- Insert 10 Doctors

```
INSERT INTO Doctors VALUES  
(1, 'Dr. Rajesh Kumar', 'Cardiology', 1000),  
(2, 'Dr. Priya Sharma', 'Neurology', 1200),  
(3, 'Dr. Anil Reddy', 'Orthopedics', 900),  
(4, 'Dr. Sneha Rao', 'Dermatology', 800),  
(5, 'Dr. Vikram Singh', 'Cardiology', 1100),  
(6, 'Dr. Ananya Das', 'Pediatrics', 700),  
(7, 'Dr. Kiran Patel', 'Neurology', 1300),  
(8, 'Dr. Meghana Devi', 'Orthopedics', 950),  
(9, 'Dr. Rohit Verma', 'Dermatology', 850),  
(10, 'Dr. Arjun Kumar', 'Pediatrics', 750);
```

-- Insert 10 Patients

INSERT INTO Patients VALUES

(1, 'Aarav Reddy', 'aarav@gmail.com'),
(2, 'Diya Sharma', 'diya@gmail.com'),
(3, 'Aditya Kumar', 'aditya@gmail.com'),
(4, 'Ishita Rao', 'ishita@gmail.com'),
(5, 'Rohan Singh', 'rohan@gmail.com'),
(6, 'Ananya Patel', 'ananya@gmail.com'),
(7, 'Varun Das', 'varun@gmail.com'),
(8, 'Sneha Verma', 'sneha@gmail.com'),
(9, 'Rahul Mehta', 'rahul@gmail.com'),
(10, 'Pooja Devi', 'pooja@gmail.com');

-- Insert 10 Appointments

INSERT INTO Appointments VALUES

(1, 1, 1, '2026-08-18'),
(2, 2, 2, '2026-08-18'),
(3, 3, 3, '2026-08-19'),
(4, 4, 4, '2026-08-19'),
(5, 5, 5, '2026-08-20'),
(6, 6, 6, '2026-08-20'),
(7, 7, 7, '2026-08-21'),
(8, 8, 8, '2026-08-21'),
(9, 9, 9, '2026-08-22'),
(10, 10, 10, '2026-08-22');

-- Display all tables

SELECT * FROM Doctors;

SELECT * FROM Patients;

SELECT * FROM Appointments;

-- INNER JOIN

SELECT

 p.patient_name,

 d.doctor_name,

 d.specialization,

 a.appointment_date

FROM Appointments a

INNER JOIN Patients p

 ON a.patient_id = p.patient_id

INNER JOIN Doctors d

 ON a.doctor_id = d.doctor_id;

-- Number of doctors in each specialization

SELECT

 specialization,

 COUNT(*) AS number_of_doctors

FROM Doctors

GROUP BY specialization;

-- Doctor History Table

```
CREATE TABLE Doctor_History (  
    history_id INT PRIMARY KEY AUTO_INCREMENT,  
    doctor_id INT NOT NULL,  
    doctor_name VARCHAR(100) NOT NULL,  
    action VARCHAR(50) NOT NULL,  
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Transaction

```
START TRANSACTION;
```

```
INSERT INTO Doctors
```

```
VALUES
```

```
(11, 'Dr. Neha Kapoor', 'Cardiology', 1050);
```

```
INSERT INTO Doctor_History
```

```
(doctor_id, doctor_name, action)
```

```
VALUES
```

```
(11, 'Dr. Neha Kapoor', 'Doctor Registered');
```

```
COMMIT;
```

-- Verify transaction

SELECT * FROM Doctors;

SELECT * FROM Doctor_History;

-- Create Index

CREATE INDEX idx_specialization

ON Doctors(specialization);

-- Search doctors by specialization

SELECT *

FROM Doctors

WHERE specialization = 'Cardiology';

```
117 • SELECT * FROM Doctor_History;
118
119 -- Create Index
120 • CREATE INDEX idx_specialization
121 ON Doctors(specialization);
122
123 -- Search doctors by specialization
124 • SELECT *
125 FROM Doctors
126 WHERE specialization = 'Cardiology';
```

Automatic o manually get h,

Result Grid

	doctor_id	doctor_name	specialization	consultation_fee
▶	1	Dr. Rajesh Kumar	Cardiology	1000.00
	5	Dr. Vikram Singh	Cardiology	1100.00
	11	Dr. Neha Kapoor	Cardiology	1050.00
•		mm	mm	mm

Export/Import,Wrap cql Contm:

Result Grid

Form Ed.,

Fwd Tbl>K

Output

		Time	Action	
0	79	15:33:28	SELECT FROM Doctor_History LIMIT 0, 1000	Mo=g•1 row(s) returned
◆	80	15:33:28	CREATE INDEX idx_specialization ON Doctors(specialization)	0 row(s) affected Records:0 Duplicates:0 Warnings:0
0	81	15:33:28	SELECT FROM Doctors WHERE specialization='Cardiology' LIMIT 0, 1000	3 row(s) returned